

School of Computer Science and Engineering, UNSW



WK03-02: Software Architecture Basics

Software Architecture for Blockchain Applications,
COMP6452

Never Stand Still

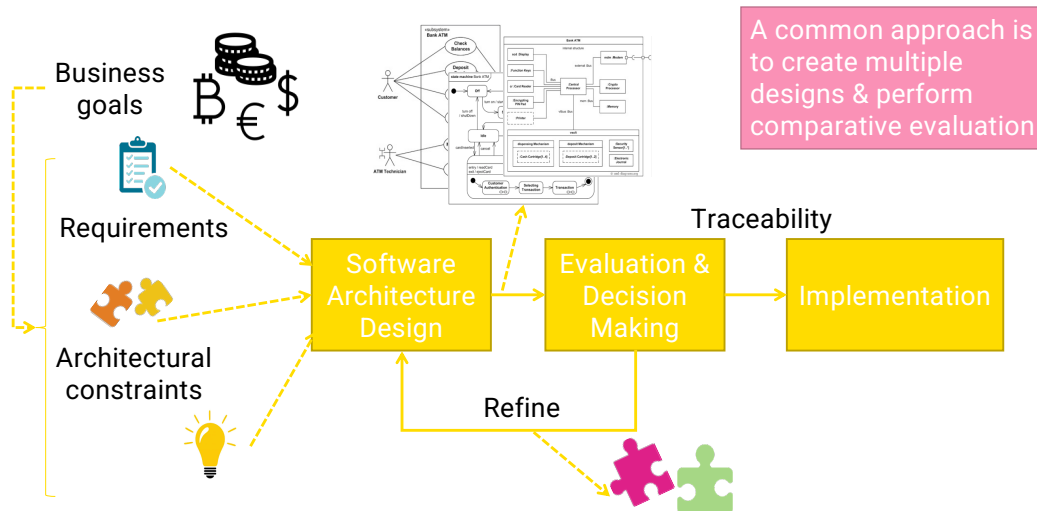
Salil Kanhere, Nadeem Ahmed

Agenda

- Software design process
- Requirements
 - Functional & Non-functional
- Software architecture
 - Viewpoints
 - Components & connectors
- Architecture analysis & design trade-offs

- We start with a discussion on the high-level software design process.
- Then we'll discuss functional and non-functional requirements that determine what and how to build.
- The third topic is the architectural aspects of software design and how to document a design.
 - In here we'll talk about different viewpoints where we look at the design from different angles.
 - Components and connectors are one of the most common ways of designing systems and depicting it visually
- Finally, we'll discuss how to analyse different architectural designs and take design trade-offs.
 - We'll briefly talk about a structured way to do this using a method called Architecture Trade-off Analysis Method (ATAM)

Software Design Process



WK 03-02

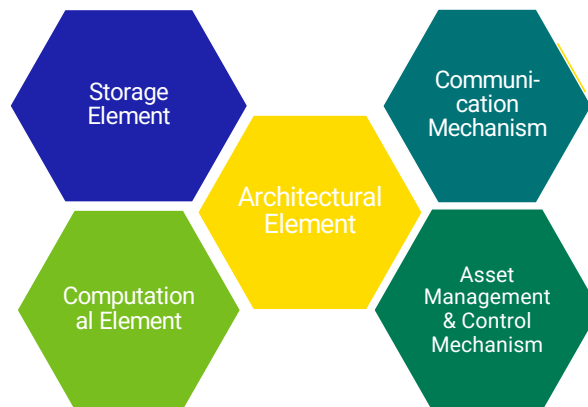
3



- This is the design process from the software architecture perspective.
 - Software design is a creative process, which includes proposing and evaluating solutions to complex problems with many conflicting constraints.
 - The final design of a software system after iterations is the result of many design choices about the selection, configuration, and integration of software, hardware, and communications components.
 - We start with a set of requirements that are based on some business goals. Business goals are ultimately driving the need for software and they provide funding.
 - We also need to deal with some architectural constraints (e.g., CAP theorem says we can have only 2 or 3 properties) and may want to try out a few ideas too (e.g., build the solution using a blockchain).
 - We then design a software architecture. The design is usually depicted as a set of diagrams with some natural language explanations/descriptions.
 - Next, we evaluate the architecture against our (non-functional) requirements. Refine multiple times if needed.
 - Then we proceed to implementation. We should be able to trace how the design translates into the implementation.
 - Usually, we develop a couple of architectures and choose the one that satisfies most of our (non-functional) requirements.
- This process is very similar to building a house. Your business need is either a place to live or investment property. You will get an architect to produce a couple of initial designs/blueprints and then compare them, not just based on appearance but functionality, practicality, cost, etc. Ultimately, you'll converge on a design that has an acceptable set of trade-offs. However, your friend going to the same architect may converge on a very different design as the trade-offs they wish to make would be different from yours (also they may have different requirements). Given that your and your friend's designs are different you will end up with very different houses too.
- "Software design" in this module is focusing on the overall system design than the user experience design (UX). Applying the analogy to a house, by design, we are not thinking about the colour of the house but more about where the rooms are located their sizes, and roof type.

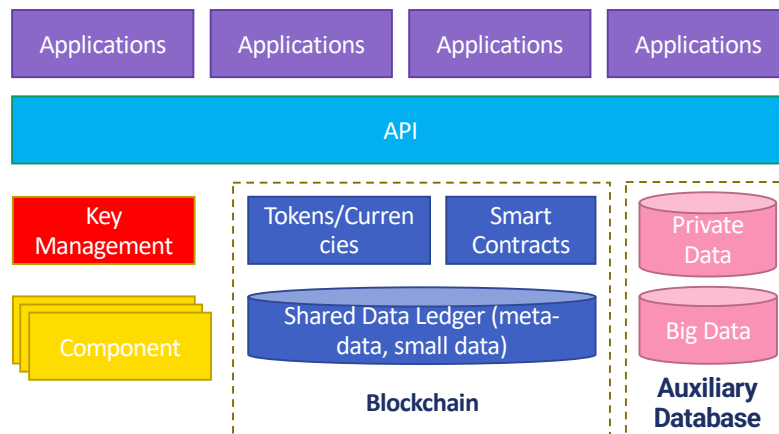
Roles Blockchain can Play in an Architecture

Blockchains are architectural design choices



- To use a blockchain in the software design process, we need to understand what it can and can't do.
 - These can feed into design options as well as architectural constraints.
- Blockchains can provide 4 key functions as an architectural element in software architecture.
- Within an architecture of a software solution, a BC can be integrated as an element that provides storage and computation.
 - Storage elements – essentially as a database.
 - Computational element – As a computing engine that can run SCs.
- It could also be used as a communication mechanism for multi-party interactions – one party writes data into the BC that the other party can see, e.g., in supply chains.
- Its storage and computation aspects can be combined to provide asset management and control mechanisms too.

Blockchain in a larger Software System



WK 03-02

5



- Once you design a software system consisting of a blockchain, it may look like this figure (this doesn't mean all systems using a blockchain will look like this).
- This is a layered design where applications are at the top and storage is at the bottom. API interconnects the applications with the storage.
 - There's a bit of abasement of the layering where key management as key management and components may be cross-cutting across layers or even sits on top of storage components. Such depictions are common to keep the diagram simple.
- The storage layer is split into 2 main components, blockchain and database.
 - The blockchain stores and shares data and executes smart contracts. The blockchain component might also control digital currency or represent other assets.
 - Due to limitations of privacy and scalability, there are also off-chain auxiliary databases used in the system.
 - First, private data is stored in an internal database. Second, large data is stored separately, such as in a cloud service.
- Key management is an essential component when working with blockchains. Every participant in a blockchain network has one or more private keys, which are used by the participant to digitally sign its transactions.

Designing & Refining an Architecture

Architecture impacts

- Performance, availability, security, privacy, scalability, maintainability, ...

But you can't achieve all in the same architecture, e.g.,

- CAP theorem says any 2 out of consistency, availability, & partition tolerance
- Transparency vs Privacy

Trade-offs must be made regarding quality attributes

- Make informed & rationale architectural decisions
- E.g., whether to use a blockchain or some conventional components should be decided
 - based on the need to enhance trust or efficiency
 - not because its a cool technology or you own crypto

- A software's architecture impact many of its properties such as performance (e.g., latency and throughput), availability, security, privacy, scalability, and maintainability.
 - These properties are known as quality attributes (more on this a bit later).
 - Therefore, design designs result in different quality attributes.
- However, you can't achieve all properties at the same time in the same architecture. E.g.,
 - If you remember, the CAP theorem says we can achieve only any 2 out of consistency, availability, and partition tolerance.
 - While blockchains enhance transparency too much transparency conflicts with privacy or business confidentiality.
- Therefore, when designing and refining an architecture we need to make design tradeoffs regarding these quality attributes.
 - We must make informed and rational architectural decisions knowing the impact of each design choice
 - E.g., we must be objective in deciding whether a certain application should or should not use a blockchain. It should be decided depending on whether we need to enhance trust and/or efficiency, not because blockchain is the state of the art or you own cryptocurrency and want to market to grow.

Agenda

- Software design process
- Requirements
 - Functional & Non-functional
- Software architecture
 - Viewpoints
 - Components & connectors
- Architecture analysis & design trade-offs

Requirements

Are specifications that describe what a software system should do & how it should behave

- Define functionality, performance, & constraints of software
- Defined at different levels of abstraction
 - Technical level, e.g., Debits always balance credits for each transaction in accounts database of software-based accounting system
 - Business level, e.g., Software-based accounting system enforces accounting standards

Serve as a foundation for entire software development process

- Must be documented, validated, & maintained
- Well-defined requirements
 - Improve communication among stakeholders, speed up development, reduce costs, enhance software quality, increase customer satisfaction

- Before designing a software system you need to know what you are designing.
- *Requirements* are the specification for your design that describes what a software system should do and how it should behave.
 - They define the functionality, performance, and constraints of a software system.
- Requirements may be defined at different levels of abstraction, e.g.,
 - Technical level, e.g., debits always balance credits for each transaction in the accounts database of a software-based accounting system.
 - Business level, e.g., software-based accounting system enforces accounting standards.
- Requirements serve as a foundation for the entire software development process.
 - What you design depends on your requirements, and what you implement on the design. So ultimately what you get depends on requirements.
 - Requirements must be documented, validated, and maintained.
 - The maintenance part is very important as requirements always change even as you design and develop the system. You can freeze requirements (e.g., waterfall model), accommodate them as they change (e.g., agile model), or anything in between. Today, the agile model is more popular meaning users prefer to accommodate requirements as they change.
 - Well-defined requirements help improve communication among stakeholders, speed up development, reduce costs (reduce changes due to unclear requirements), enhance software quality, and most importantly increase customer satisfaction.

Functional & Non-Functional Requirements

Functional Requirements (FRs)

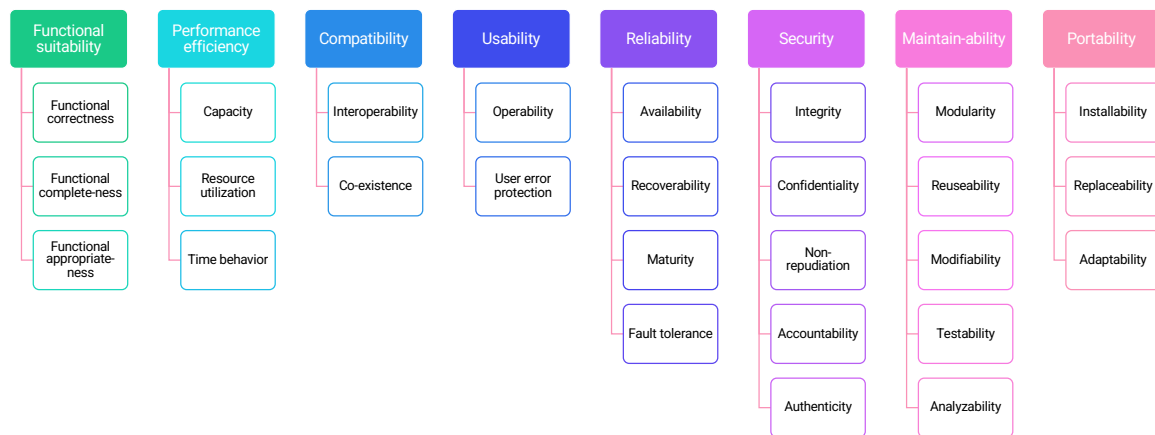
- Describe a system's relationships between inputs, internal states, & outputs

Non-functional requirements (NFRs)

- Aka non-functional properties, software qualities, or –ilities
- Describe how systems behave in other ways, e.g., speed or user friendliness
 - E.g., accounting system need to process 250 TXs/sec & each transaction within a 1.5 Sec
- Many quality characteristics
 - Performance (latency, throughput, ...)
 - Security (confidentiality, integrity, availability, privacy, ...)
 - Usability, reliability, modifiability, ...

- Requirements are classified as functional requirements (FRs) and non-functional requirements (NFRs).
- FRs describe a system's relationships between inputs, internal states, and outputs.
 - What the system should do.
- NFRs describe how systems behave in other ways, e.g., speed or user-friendliness.
 - E.g., the accounting system needs to process 250 TXs/sec & each transaction within a 1.5 Sec
 - Are also known as non-functional properties, software qualities, or –ilities
 - NFRs are essential to ensure a system is practically useful. So it's insufficient to achieve only the FRs.
 - There are many quality characteristics. Some of the frequently occurring software qualities are:
 - Performance (latency, throughput, ...)
 - Security (confidentiality, integrity, availability, privacy, ...)
 - Usability, reliability, modifiability,

ISP/IEC 25010:2011 Quality Model



- International standards for evaluation of software quality, ISO/IEC 25010:2011 Quality Model, define many software qualities. It is expected that this standard will be revised soon, but it was reconfirmed in 2017.
- Other than functional suitability all other qualities are non-functional. Under each group there are multiple quality metrics.
- Blockchain has good and bad qualities.
- We'll mostly focus on the functional suitability, performance efficiency, reliability, and security of BC-based applications in this module. We'll define selected these metrics in later classes.

[illegible]

- 11

Documenting Requirements

Best practices

- Ensure requirements are specific, detailed, & complete
- Use clear & concise language
- Use standard notations to enhance clarity, e.g., use case & UML diagrams
- Requirements should be easy to verify & validate
- State assumptions

Examples

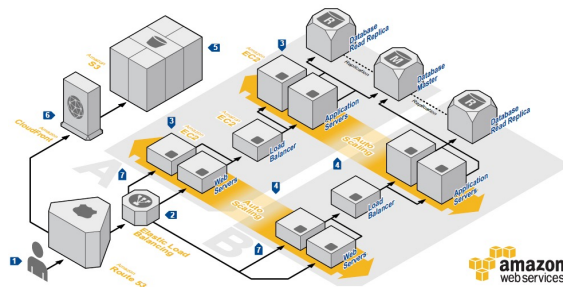
- FR.5 Consent – The system **SHOULD** allow the data requesters to request consent to access data from the data owner.
- NFR1.1 Confidentiality – The system **MUST** ensure that data are accessible only to authorised parties per the approvals and consent.

- As requirements determine what we design and develop, we must document them precisely.
- There are many best practices when it comes to documenting requirements. The following are some of them:
 - Ensure that requirements are specific, detailed, and complete.
 - Use clear and concise language. Write requirements using clear, unambiguous language that is easily understandable by all stakeholders. Avoid jargon, technical terms, and ambiguous statements that can lead to misinterpretation.
 - Utilize standard notation, such as use case diagrams, flowcharts, or UML diagrams, to enhance clarity and understanding. Visual representations can help stakeholders better visualize and comprehend complex requirements.
 - Write requirements in a way that allows for easy verification and validation. Make requirements testable by specifying acceptance criteria, input data, expected outputs, and success criteria.
 - Conduct reviews and validation sessions with stakeholders to ensure that the documented requirements accurately capture their needs. Incorporate feedback and make necessary revisions to improve the quality of the requirements.
 - Include any assumptions made during requirement gathering and note any constraints or limitations that may impact the requirements.
 - Use a structured format for documenting requirements, such as a table or a template. Include sections for requirement ID, description, priority, source, and any other relevant attributes.
 - Provide the necessary context and background information for each requirement. Explain the purpose, business value, and underlying rationale behind the requirement.
 - Requirements change. To implement a version control system to manage changes to the requirements document. Regularly update the requirements document as changes occur throughout the software development lifecycle.
- Here are 2 examples of requirements:
 - FR.5 indicates that is the 5th functional requirement. We have also given it a label called “Consent” as the requirement is about consent.
 - NFR1.1 indicates is the 1st sub-requirement under security and it’s labelled “confidentiality”
 - Note the use of **SHOULD** and **MUST** in the requirements which are keywords from <https://www.ietf.org/rfc/rfc2119.txt>
 - **MUST** means that the definition is an absolute requirement of the specification.
 - **SHOULD** means that there may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.

Agenda

- Software design process
- Requirements
 - Functional & Non-functional
- Software architecture
 - Viewpoints
 - Components & connectors
- Architecture analysis & design trade-offs

Software Architecture



A blueprint

Overall design of a software system,
including

- Elements or components of software
- Their connections
- Their deployment onto hardware

Captures a set of principal design
decisions about a system

- Software architecture is the blueprint of a system.
- There are several definitions of software architecture. But we can go with the simple definition that says, software architecture is the overall design of a software system, including
 - Individual elements or components of the software, e.g., database
 - Their connections, e.g., ODBC connection
 - Their deployment onto hardware, e.g., database server
- This is a generic architecture diagram of a 3-tier web application hosted on AWS (Amazon Web Services).
https://aws.amazon.com/architecture/?cards=all.sort-by=item.additionalFields.sortDate&cards-all.sort-order=desc&awsf.content-type=*all&awsf.methodology=*all&awsf.tech-category=*all&awsf.industries=*all&awsf.business-category=*all
 - A diagram like this is called a component-connector diagram as it shows components and their connectivity.
 - It has presentation (web servers), business logic (application servers), and storage layers (databases).
 - This diagram has many components such as DNS load balancers, application load balancers, web servers, application servers, primary and secondary databases, and storage.
 - Arrows between those components are the network connectors.
 - Their hardware mapping is also indicated, e.g., web and application servers run as EC2 virtual machines (VM). Storage is on AWS S3 (Simple Storage Service).
- Software architecture captures a set of principal design decisions about a system. E.g., this diagram depicts that the system is based on the 3-tier architecture and designed for high availability (note redundant components).
 - However, it is important to note that not all decision decisions are architectural.

Architecture is an Abstraction

Architecture provides an abstract view of a design

- Hides complexity of design & enhances communication
- May or may not be a direct mapping between architecture elements & software elements
 - c.f., “marketecture”

Models are abstractions; What to model & what to abstract depends on the purpose

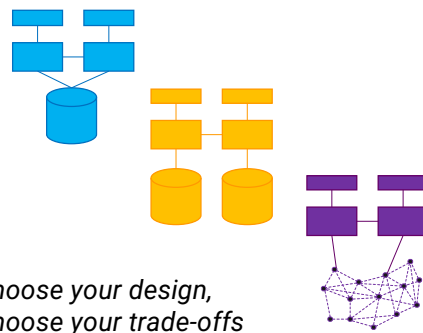
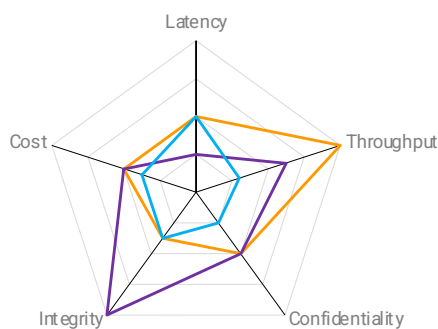
- *“All models are wrong, but some models are useful”* – George Box
- *“[A] model which took account of all the variegation of reality would be of no more use than a map at the scale of one to one”* – Joan Robinson, 1962

- Architecture is an abstraction/blueprint of the software system’s design.
- It hides the complexity of the design and makes discussion/communication between different stakeholders easy.
- There may or may not be a direct mapping between architecture elements and software elements. It’s never correct or wrong. It’s about whether the architecture is fit for the purpose.
 - A *Marketecture* (aka Marchitecture) is a non-technical description of a system, its structures and interactions. It is used to help customers, partners, and investors understand what the product/system does and how it fits into a broader ecosystem.
- While it may not be a perfect mapping of the system to be built it gives something useful to documentation and communication.
- It can be drawn at different levels of abstraction targeting different stakeholders.
- Architectures are reviewed (sometimes by 3rd parties) to determine how well they satisfy NFRs. Hence, they should be able to understand what is documented/depicted.

Software Architecture & Non-Functional Properties

Non-functional properties arise from architectural design choices

Create multiple architectural designs & choose the one that satisfies significant non-functional properties



*Choose your design,
Choose your trade-offs*

WK 03-02

16



- There is a close relationship between software architecture and the non-functional properties it achieves.
- Non-functional properties arise from architectural design choices. Hence, an architecture is designed to achieve most NFRs.
- Here are 3 designs based on 3-tier architecture.
 - The first design uses 2 presentation and application servers. However, it has only a single database leading to a single point of failure.
 - The second design replicates all 3 layers. However, synchronising parallel databases is quite complex (recall discussion from 1st lecture). Also, 2 databases mean more cost.
 - The third design replaces the database with a blockchain. This takes awareness of the synchronisation issues that need to be handled by the developer. However, blockchain tends to be a slower and more expensive option than a database.
- So choose your design based on the trade-offs you are willing to make.
- A typical practice is to create multiple architectural designs and then choose the one that satisfies significant non-functional properties. This is mostly a qualitative evaluation.
 - So by choosing one design over another, we are choosing various NFR trade-offs.

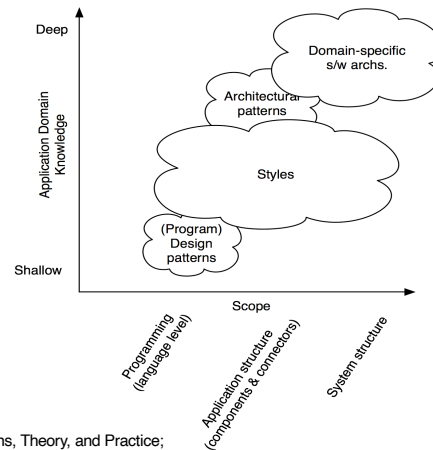
Software Architecture Design

From scratch

- Unexpected solutions can be found
- Labour-intensive & error-prone

Apply a generic solution or strategy (style/pattern) & adapt it to the problem

- Reuse, less work, & less errors
- Generic solution might be ill-fitting or too generic



Source: Software Architecture: Foundations, Theory, and Practice; Richard N. Taylor, Nenad Medvidovic, and Eric M. Dashofy; 2008, John Wiley & Sons, Inc.

WK 03-02

17



- There are two ways to design a software architecture.
- One is from scratch.
 - When you take your approach, you may (rarely) find unique/unexpected solutions to a problem.
 - However, this process is labour-intensive and error-prone, and the solution might be suboptimal.
- The other way is to apply a more generic solution or strategy, like the architectural style or architecture pattern, and adapt it to the problem like we use libraries in programming.
 - Reusing existing design from others cause us less work and fewer errors.
 - However, there's also a risk that the generic solution might be ill-fitting or too generic to solve your specific problem.
- The diagram gives two dimensions for the classification of some architectural design concepts.
 - The x-axis represents scope, which is from the programming language level to the system structure level.
 - The y-axis represents application domain knowledge, which is from shallow to deep.
 - The design patterns you are perhaps familiar with are at the language level and relatively shallow in terms of domain knowledge. Which means it is agnostic to the domain.
 - The range of architectural styles can be large, which covers language level, application structure, and system structure.
 - Architecture patterns, on the other hand, are at application structure, which doesn't give a system-level structure.
 - Finally, domain-specific software architectures need the deepest domain knowledge and give a system-level design solution.
- A design pattern is a recurring solution to a common problem related to implementation, e.g., factory pattern and proxy.
- Architectural style is a recurrent architectural design that shows how we organise systems. It doesn't solve a specific problem, e.g., client-server, peer-to-peer, microservices architecture.
- An architectural pattern solves a specific architectural problem, e.g., a 3-tier design is an architectural pattern that is used with client-server architecture style to separate a system into 3 layers, namely presentation, application logic, and data.
- A single architecture can contain several Architectural Styles, and each Architectural Style can make use of several Architectural Patterns. An Architecture Pattern can be a subset of an Architectural Style targeting a specific scope.

Architectural Styles vs Patterns

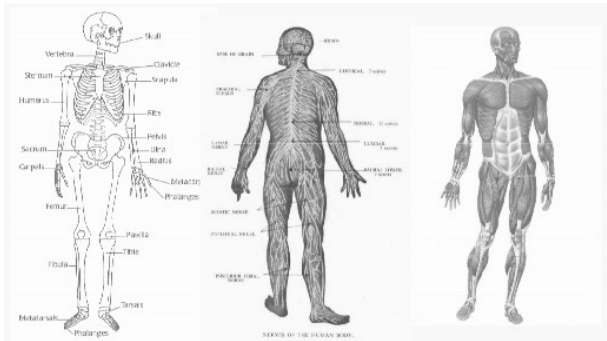
Architectural Style	Architectural Pattern
A set of principles & guidelines that dictate overall characteristics of the architecture	A general, reusable solution to a recurring design problem
Defines vocabulary, design principles, & constraints	Set of predefined rules, principles, & relationships among components
Doesn't solve a specific problem	Solves a specific problem
May encompass multiple architectural patterns	Typically independent
Client-server, Peer-to-peer Service Oriented Architecture (SOA) REpresentational State Transfer (REST) Object-oriented Publisher-Subscriber	3-tier Model-View-Controller (MVC) Microservices Extract, Transform, Load (ETL)

- Architectural styles and patterns are used interchangeably. While some authors distinguish the two, others don't care.
 - You can think of Romanesque, Gothic, and Tudor as architectural styles while doors, windows, and ceilings as patterns.
- An architectural pattern is a specific solution to a design problem, while an architectural style represents a higher-level concept that defines the overall characteristics and qualities of an architecture. Here's one set of definitions we can work with.
- Architectural Style
 - Is a set of principles and guidelines that dictate the overall characteristics and qualities of the architecture.
 - It defines the vocabulary, design principles, and constraints for building software systems.
 - An architectural style focuses on the broader aspects of the system, such as the overall structure, communication protocols, and coordination mechanisms.
 - It may encompass multiple architectural patterns and defines how they fit together.
 - Common architectural styles include Client-Server, Peer-to-Peer, Service-

Oriented Architecture (SOA), and Event-Driven Architecture.

- Software Architectural Pattern
 - Is a general, reusable solution to a recurring design problem in software architecture.
 - It represents a high-level abstraction that guides the overall structure and organisation of a software system.
 - Architectural patterns provide a set of predefined rules, principles, and relationships between components or modules to address common design challenges.
 - Examples of architectural patterns include the Model-View-Controller (MVC), Layered Architecture, Microservices, and Event-Driven Architecture.
- 3-tier is a pattern used in the client-server style.
- Microservices is a pattern used in SOA that designs a system as a set of services.

Viewpoints & Views



- View – Representation of a specific set of architectural concerns
- Viewpoint – Perspective, concerns, & guidelines for creating architecture views

- We view a software architecture from different viewpoints.
- An architectural *view* is a representation of a specific set of architectural concerns, while a *viewpoint* defines the perspective, concerns, and guidelines for creating architectural views.
- Architecture views are tangible outputs that capture a slice of the architecture, while viewpoints provide the context and framework for creating those views.
- E.g., these are different views of a human's anatomical model, including the muscular model, circulatory model, and skeleton. These 3 models describe the human body from different viewpoints.
 - The diagram on the right-hand side of the slide describes a human's hand from a skeleton viewpoint.
- E.g., the Developer and system admin need different views of the same system, as their concerns are different. So develop viewpoint say architecture should be documented based on modules, classes, and their interactions. Whereas the system admin is interested on hardware it runs and services like databases.
- Similarly, we have the blockchain network view that is different from the ledger state view. The former focus on nodes in the network while the latter focuses on data on the ledger. We have have smart contract view that depicts the interactions among smart contracts.

Krutchchen's 4+1 View Model

Logical – Architecturally-significant elements & their relationships

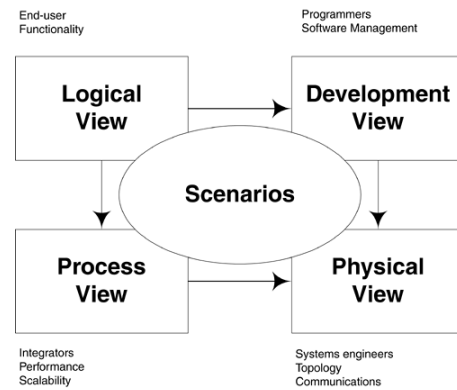
Process – Dynamic behaviour, concurrency & communications elements

Physical – How processes & components are mapped to application's hardware

Development – Internal organisation of software components

Scenarios/Use cases – Concrete examples of system behaviour in specific contexts

- Related to more than 1 view



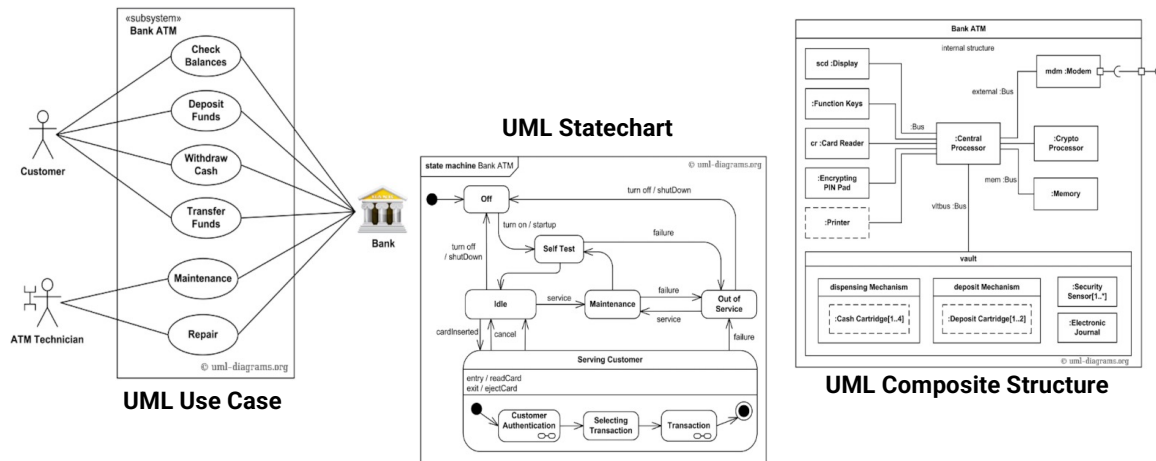
WK 03-02

20



- Krutchchen's 4+1 View Model, by Philippe Kruchten, is a popular architecture documentation technique to view a system from 5 viewpoints.
- It provides a holistic approach to capturing different aspects of a software system's architecture by utilising multiple architectural views.
- The "+1" in the model refers to the use of scenarios or use cases as an additional view to complement the other four views.
- **Logical View**
 - Focuses on architecturally significant elements and their organisations to achieve functional requirements of the system.
 - It captures the key abstractions, modules, classes, etc., and their relationships.
 - It emphasises the software's internal structure, interactions, and behaviour, abstracting away implementation details and technology-specific aspects.
- **Process View**
 - Emphasises the dynamic behaviour of the system and how it is executed.
 - It captures the system's runtime behaviour, concurrent processes, tasks, and their interactions.
 - The process view provides insight into the system's performance, scalability, and resource utilisation.
- **Physical View**
 - The physical view emphasises the mapping of the software architecture onto the hardware and deployment infrastructure.
 - It captures the distribution of components, nodes, servers, and communication channels.
 - It considers systems engineering aspects such as availability, deployment topology, and physical constraints.
- **Development View**
 - Focuses on the organisation of the software components during the development process.
 - It captures the modular structure, subsystems, and development dependencies.
 - The development view helps understand how the system is partitioned, managed, and evolved by development teams.
- **+1. Scenarios**
 - Scenarios or use cases complement the other four views by providing concrete examples of system behaviour in specific contexts.
 - Scenarios illustrate how the system interacts with external entities, users, and other systems.
 - They help validate and demonstrate the architectural decisions made in the other views.
- By employing Krutchchen's 4+1 View Model, software architects can address various perspectives in a comprehensive manner, enabling effective communication and documentation of the system's architecture to different stakeholders.

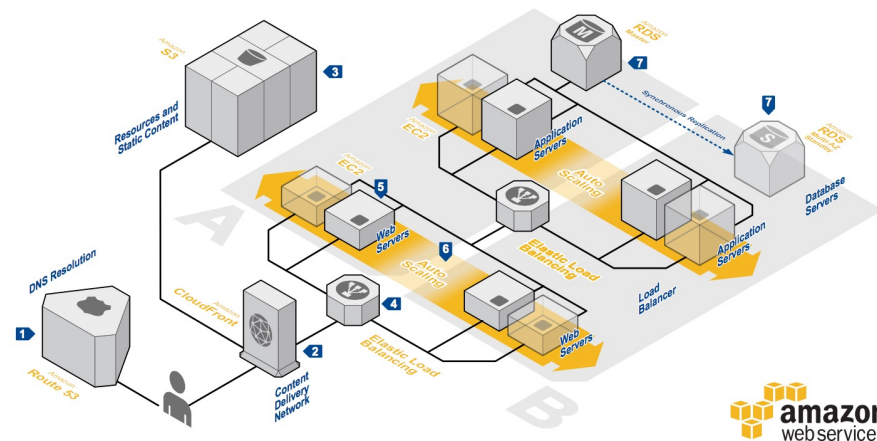
UML as Viewpoints & Views



- Different UML diagrams to describe the system from different views, e.g.,
- Use case diagrams to present scenarios in Krutchen's 4+1 View that describe a customer's and ATM technician's interaction with an ATM.
- UML Statecharts show the process view as it depicts the dynamic behaviour of an ATM.
- UML composite structure shows the internal structure of a [class](#) and the collaborations that this structure makes possible. Therefore, it focuses on the development view.

Software Architecture's Elements

Component
Connector
Configuration



amazon
web services

WK 03-01

22

UNSW
AUSTRALIA

- A software system's architecture typically is not a uniform monolith.
- As seen from a real-world software architecture of a typical web application architecture from the AWS architecture reference centre, we see components, connectors, and configurations.
 - A software component is an architectural building block that encapsulates a subset of the system's functionality and/or data and restricts access to them via an explicitly defined interface. Various shapes of 3D objects we see in the figure are components.
 - A software connector is an architectural building block tasked with regulating interactions among components. Lines are connectors.
 - Architectural configurations, or topologies, capture architectural structure as graphs whose nodes represent components and connectors, and whose edges represent their interconnectivity. The layout we see in the figure is the configuration.
- A bit of detail about the diagram
 - Users need DNS resolution to find the IP address of the web application.
 - The web app is a 3-tier architecture. Both the application layer and web layer are deployed in an auto-scaling group, and connected with a load balancer each, which can balance the requests received from the end users.
 - Static content of the web app is stored on the side in S3, which is an AWS storage service.

Software Component

A fundamental building block of software architecture

An architectural entity that

- Encapsulate a subset of system's functionality or data
- Restricts access to that subset via an explicitly defined interface
- Has explicitly defined dependencies on its required execution context

Modular, self-contained, & reusable

Components typically provide application-specific services

Blockchain as a storage & computational element

- Software connectors are the fundamental building blocks of a software architecture. These are the Lego blocks.
- A software component is an architectural building block that encapsulates a subset of the system's functionality and/or data and restricts access to them via an explicitly defined interface.
- Also, has explicitly defined dependencies on its required execution context.
 - E.g., a database can run only on a specific operating system with a given hardware spec.
- Thus, it is a modular, self-contained, and reusable unit of software that encapsulates a set of related functionalities.
- However, components typically provide application-specific services.
- Blockchain can be used as a storage and computational element/component in an architecture to store data and execute business logic, respectively.

Software Connector

Complex systems need to integrate functionality from multiple components

Architectural building block tasked with effecting & regulating interactions among components

- Glue that enables components to work together

Typically provides application-independent interactions

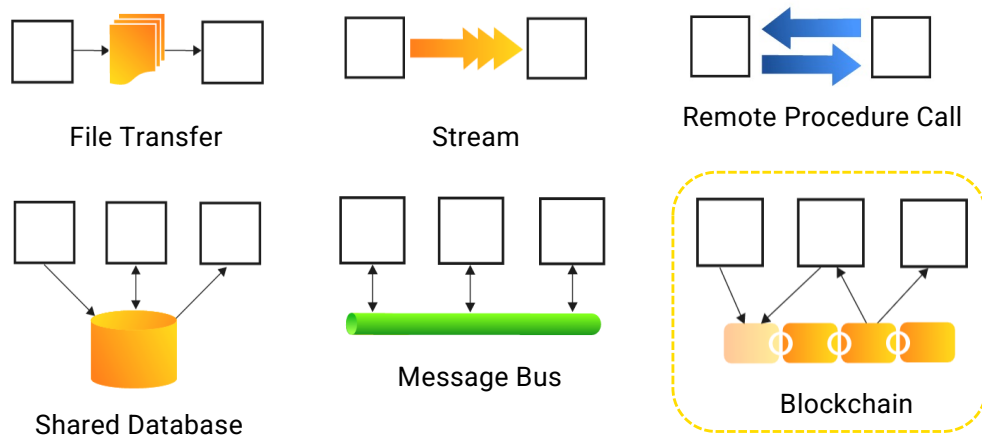
- Communication – transfer data
- Coordination – transfer control
- Facilitation – enable & optimise component's interactions
- Conversion – Adjust interactions between incompatible interfaces

- A software system needs to integrate functionality from multiple components. In complex systems, such interactions may become more important and challenging than the functionality of the individual components.
- A software connector is an architectural element that facilitates communication and interaction between software components within a larger software system.
- Connectors serve as the glue that enables components to work together and exchange information, allowing for the coordination and cooperation of different parts of the system.
- Connectors promote loose coupling and flexibility in the system's architecture, where they allow one component to be replaced with another without significant impact to the interaction (other properties may be affected).
- Connectors typically provide application-independent interaction facilities such as:
 - Communication – transfer data from one component to another. A network connector at the hardware level. At the software level, FTP can be considered as a connector as it allows a file to be transferred from one system to another. They define the communication protocols, formats, and mechanisms used for interaction, such as method invocations, function calls, event notifications, or message passing.
 - Coordination – transfer control from one component to another. In data

migration, first data are extracted, then transformed and load into the new system (called ETL process). These 3 steps need to be coordinated.

- Facilitation – enable and optimise component interactions. E.g., a load balance split load among multiple connectors. Web3.js can validate transactions. Another example is remote-procedure calls can be simplified using various libraries.
 - Conversion – Adjust interactions between incompatible interfaces. E.g., we may use a connector to translate data from XML to JSON and vice versa. Useful in interconnecting incompatible systems. ODBC provides a database-independent way to access data from an application.
 - A single connector may provide multiple interaction facilities, e.g., Web3.js allow communication between an application and Ethereum, validates transactions, listens to events, etc.
- Connector != Component

Example Software Connectors

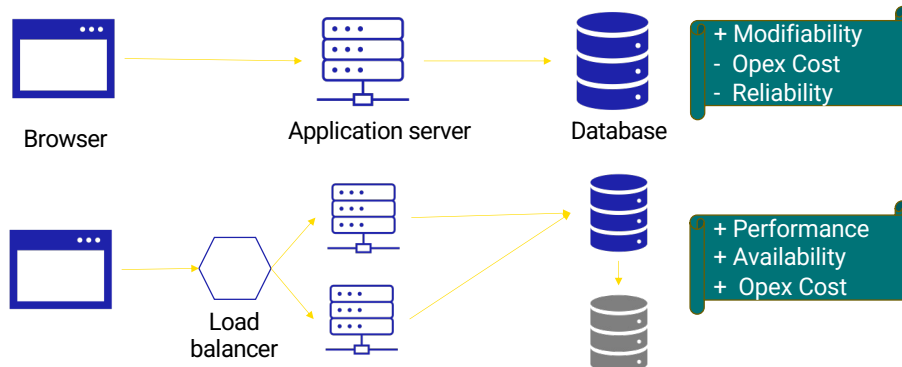


- File transferred from one component to another, e.g., FTP.
- Stream coming from one component to another.
- Simple procedure calls.
- Shared data accesses, e.g., ODBC or JDBC.
- Message bus that connects multiple components. Anyone can read/write from/to the bus.
- Much more sophisticated and complex connectors are possible: blockchain. More on this later.

Software Configuration

Aka topology

Set of specific associations between components & connectors of a system's architecture to accomplish its requirements



WK 03-02

26

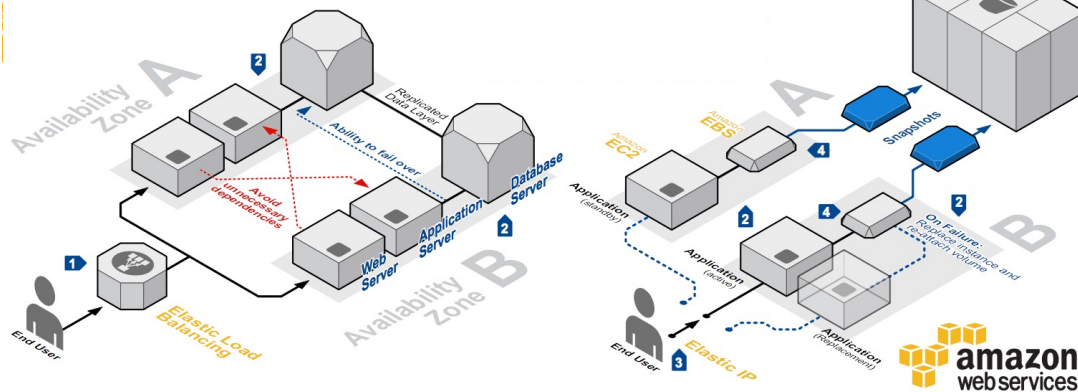


- In the context of architecture discussion, software configuration or topology is a set of specific associations between components and connectors of a system's architecture to accomplish its requirements.
 - This is not quite the configuration options such as database connection details, network settings, feature toggles, logging levels, or performance tuning parameters.
- Here's an example of a 2-tier architecture configuration.
- Given the same components, we can build a different configuration as seen in the 2nd figure. We have 2 new connectors too.
 - The load balancer is considered a connector, not a component as it mediates the communication between web and application servers.
- Two design alternatives with different qualities.
 - Single web and database server has good modifiability, and lower operation cost, but low reliability at the same time.
 - Add one more web server to the architecture and a load balancer in front of the 2 web servers. A hot replica of the database in the data layer. This improves performance and availability at the expense of high operational costs.

AWS Fault Tolerance & High Availability

High availability – Minimise & eliminate downtime

Fault tolerance – Continue to operate despite failures/faults



WK 03-02

27

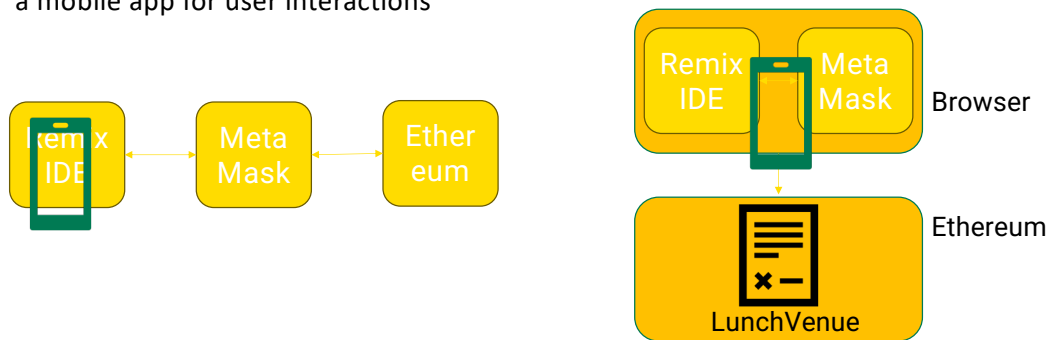


- This slide gives two other architecture diagrams from the AWS architecture reference centre.
- The design on the left is aimed to provide better fault tolerance and the one on the right is aimed to provide high data availability.
 - High availability and fault tolerance are both approaches to ensure system reliability, but they differ in their goals and methods.
 - High availability aims to minimise and eliminate downtime and ensure continuous accessibility of a system, while fault tolerance focuses on the system's ability to operate correctly despite failures or faults.
- The design on the right (high availability design) has passive redundant components that can take over after a failure.
 - Valuable data should never be stored without proper backups, replication, or disaster recovery. Therefore, the application server is connected to Amazon Elastic Block Store (EBS) and offers persistent storage, which is automatically replicated within a single availability zone.
 - To enhance data durability, point-in-time snapshots are created once in a while to store data on volumes in Amazon S3.
 - In case of failure, the application server can be replaced and reattached storage volume.
 - During the process, a standby takes over to serve requests.
 - Using a snapshot, the data stored in the persistent store can be recreated.
 - There will be a short downtime between switching from primary to secondary and returning back.
- Whereas the design on the left (fault tolerant design) has active redundant components, where the application can continue to function despite a few failures.
 - Web servers deployed in multiple availability zones.
 - Availability zones (AZs) are distinct geographical locations that are engineered to be insulated from failures in other AZs. Remove a technical single point of failure.
 - Load balancing can balance across instances in multiple availability zones. Instances that fail can replace seamlessly behind the load balancer.

Exercise

Draw a component & connector diagram for LunchVenue application (from Lab 1) deployed on Ethereum

Your friends are finding it difficult to use Remix IDE. Draw a new diagram that uses a mobile app for user interactions



- The diagram on the left shows the 3 components, their connections, and linear configuration.
 - Note the directions of arrows as data flow is bidirectional.
- The diagram on the right is also correct. We even depict the contract.
- However, 2nd one has an advantage as it shows that if you want to make user interaction easy, you can think about replacing the web browser interactions with something more user-friendly. That mean mobile app should integrate MetaMask's functionality too.

Agenda

- Software design process
- Requirements
 - Functional & Non-functional
- Software architecture
 - Viewpoints
 - Components & connectors
- Architecture analysis & design trade-offs

Models Enable Analysis

Can answer questions like

- What is transaction latency in this design?
 - Simulation-based prediction of latency distributions (Quantitative)
 - Formula-based calculation of average latency (Quantitative)
- Does this design support service availability?
 - Failure scenarios (Qualitative)
 - Fault-tree statistical analysis (Quantitative)
- Will this design ensure confidentiality?
- Is this design easily modifiable?

Different kinds of models allow different kinds of analyses

- An architectural model enables us to answer various questions such that we can analyse non-functional properties.
- E.g., we ask questions like What is transaction latency in this design?
 - This question can be answered quantitatively by either simulating the behaviour to predict the latency distributions or using formula-based calculations to estimate average latency. However, to do this architect needs to understand how such simulation work or calculations are performed, as well as typical parameters and values from similar systems.
- Another question is does this design support service availability?
 - This can be qualitatively determined by analysing various failure scenarios. E.g., from slide 27, we see there is only a single web or database server. Hence, they are the weakest point that is likely to fail. Given the same failure scenario, 2 server design will have better availability as the application can continue to operate even with a single failure.
 - The fault-tree analysis is a statistical technique that looks at the interaction among components as a tree and uses known failure statistics to determine what is likely to fail first or when it is likely to fail.
- Other related questions are:
 - Will this design ensure confidentiality? This is mostly a qualitative answer though some formal proof can be given for specific attack scenarios.
 - Is this design easily modifiable? This is mostly a qualitative answer though there are some metrics to estimate the modifiability of a code.
- As different models depict different aspects of a system, they allow different kinds of analysis.

Design Trade-offs

Often, improving one quality will hurt another, e.g.,

- Use a bigger server → faster, but more expensive
- Use redundant servers → better availability, performance (load balancing), but high write latency & more expensive

What trade-offs & how to choose them?

- Determine if architecture can achieve NFRs before developing a system (which is typically a big investment)

Specific methods exist to help

- Architecture Trade-off Analysis Method (ATAM)
- Multi-criteria decision-making methods

- Often, improving one software quality will hurt another one, e.g.,
 - By using a bigger server with more computing power and memory we can increase the execution speed. But it comes at a higher cost.
 - By using redundant servers we can enhance availability (especially if all replicas are active) and performance (as now load can be split across multiple servers). But more servers means we have to slow down writing to ensure there's enough time to sync data across multiple databases (recall blockchains?). Also, more servers come at a much higher cost.
- So given different possibilities we need to decide on what trade-offs to make and how to choose among them.
 - And we want to be able to decide these at the design stage as anything later means more changes and costs.
- There are a couple of methods that exist to help in deciding what trade-offs to make and how to choose among them.
 - ATAM: Architecture Trade-off Analysis Method. Next, we'll discuss this technique at a high level
 - Multi-criteria decision-making methods - are a sub-discipline of operations research that explicitly evaluates multiple conflicting criteria in decision-making.

Design Decisions & Their Impact

Design Decision	Option	Fundamental properties	Impact		
			Cost efficiency	Performance	Flexibility
Data	On-chain	Embedded in transaction (Bitcoin)	⊕	⊕	⊕⊕
		Embedded in transaction (Public Ethereum)	⊕⊕⊕⊕	⊕	⊕⊕⊕
		Smart contract variable (Public Ethereum)	⊕⊕	⊕⊕⊕	⊕
		Smart contract log event (Public Ethereum)	⊕⊕⊕	⊕⊕	⊕⊕
	Off-chain	Private / Third party cloud	⊕	⊕⊕⊕⊕	⊕⊕⊕⊕
		Peer-to-Peer system	⊕⊕⊕⊕	⊕⊕⊕	⊕⊕⊕
Computation	On-chain	Transaction constraints	⊕⊕⊕⊕	⊕	⊕
		Smart contract			
	Off-chain	Private / Third party cloud	⊕	⊕⊕⊕⊕	⊕⊕⊕⊕

WK 03-02

32



- Here's an example of a design decision regarding on-chain and off-chain data storage and their impact.
 - These can be some arguments about this table. As technology has evolved, let's interpret the table than worrying about whether a cell is perfectly reflected.
- We can store data on the blockchain (on-chain) or off the blockchain (off-chain).
- We evaluate these design trade-offs against their impact on fundamental properties of blockchains (e.g., immutability and integrity), cost, performance, and flexibility.
- When it comes to storing data on-chain, we can do it in a couple of ways.
 - Data can be embedded in transactions. As transactions are immutable recorded data are also immutably recorded. We are considering both Bitcoin and Ethereum to show that they can do this to different extents.
 - We can store data on smart contract variables, e.g., our voters, restaurants, and lunch venue.
 - Data can also be included in event logs as a result of executing a smart contract function.
 - These options have different pros and cons. Storing data as a variable in a smart contract is more efficient to manipulate, but less flexible due to the constraints of Solidity on the value types and length.
- Alternatively, storing data off-chain enhances cost efficiency, performance, and flexibility.
 - But these come at the loss of blockchain properties.
- Similarly, we can perform computation on and off-chain leading to different trade-offs.

Architecture Trade-off Analysis Method



A systematic approach for analysing trade-offs among different architectural design decisions

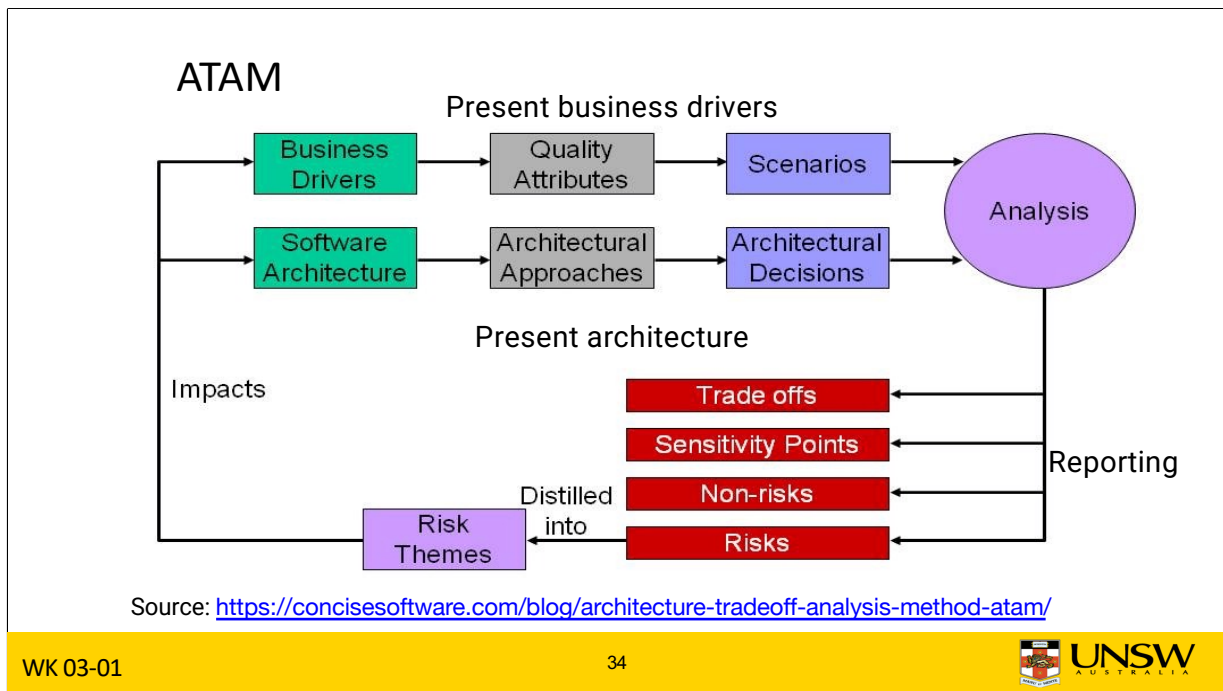


A framework for making informed decisions by considering risks associated with different architectural alternatives



Focuses on understanding trade-offs among conflicting quality attributes, identifying potential risks, & their mitigation strategies

- The Architecture Trade-off Analysis Method (ATAM) is a structured and systematic approach for evaluating and analysing the trade-offs among different architectural design decisions.
- ATAM is a risk-driven method that helps architects and stakeholders assess the potential impacts of architectural choices on various quality attributes and system goals.
- It provides a framework for making informed decisions by considering the risks associated with different architectural alternatives. It focuses on understanding the trade-offs between conflicting quality attributes (e.g., performance vs. maintainability or security vs. usability), and identifying potential risks and their mitigation strategies.



- There are different versions of this diagram.
- At the top, we present both the business goals/drivers and proposed architecture.
- Then an extensive investigation and analysis is conducted.
- Next, findings from the analysis are reported.
- These findings are then analysed based on the risks. One where risk is not tolerable architecture, business goals, or both need to be revised and reevaluated.

ATAM Steps

Presentation

1. Present ATAM to stakeholders
2. Present business drivers (NFRs & business goals)
3. Present current architecture

Investigation & analysis

4. Identify architectural approaches
5. Generate quality attribute utility tree (& initial use case scenarios)
6. Analyse architectural approaches (for high-priority factors from step 5)
7. Brainstorm & prioritise scenarios
8. Analyse architectural approaches (using scenarios as test cases)

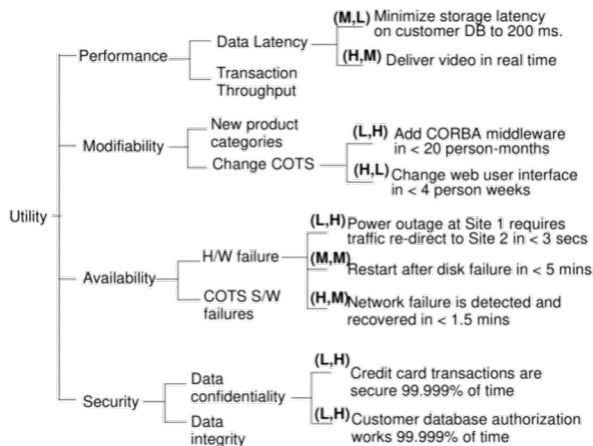
Reporting

9. Present results (summary, risks, sensitivities/trade-offs, ...)

The detailed ATAM process has 9 steps that can be split into 3 groups:

1. **Present ATAM to stakeholders** – This step calls for the architecture evaluation leader to present the ATAM to the assembled project representatives.
 2. **Present business drivers** – In this step, a project decision maker (ideally the project manager or product owner) presents a system overview from a business perspective.
 3. **Present current architecture** – The lead architect (or architecture team) then makes a presentation describing the architecture at an appropriate level of detail.
 4. **Identify architectural approaches** – As ATAM focuses on analysing an architecture by understanding its architectural approaches, adopted architectural patterns and tactics are analysed to understand how they affect particular quality attributes. For example, a layered pattern tends to bring portability and maintainability to a system, possibly at the expense of performance.
 5. **Generate quality attribute utility tree** – The quality attribute goals are articulated in detail via a quality attribute utility tree (see next slide). Utility trees serve to make the requirements concrete by defining precisely the relevant quality attribute requirements that the architects were working to provide.
 6. **Analyse architectural approaches** – The evaluation team examines the highest-ranked scenarios (as identified in the utility tree) one at a time; the architect is asked to explain how the architecture supports each one.
 7. **Brainstorm & prioritise scenarios** – The evaluation team asks the stakeholders to brainstorm quality attribute scenarios that are operationally meaningful concerning the stakeholders' roles.
 8. **Analyse architectural approaches** – After the scenarios have been collected and prioritized in Step 7, the evaluation team guides the architect in the process of analysing the highest-ranked scenarios. The architect explains how architectural decisions contribute to realising each scenario.
 9. **Present results** – The evaluation team convenes and groups risk into risk themes, based on some common underlying concern or systemic deficiency.
- This is given for completeness. It is sufficient if you understand the abstract diagram in the previous slide.

ATAM Quality Attribute Utility Tree



- Utility trees make system goals more specific & concrete
- Allows understanding relative importance of NFRs → deciding on priority

Source: Example from “ATAM: Method for Architecture Evaluation” (Kazman et al., 2000)

https://resources.sei.cmu.edu/asset_files/TechnicalReport/2000_005_001_13706.pdf

Legend: (Importance, Risk to Achieve)

- A utility tree is a top-down representation of what you, as an architect, believe to be the quality-related architecturally significant requirements that are critical to the success of the system.
 - A utility tree begins with the word “Utility” as the root node. Utility is an expression of the overall “goodness” of the system.
 - Then list the major quality attributes that the system is required to exhibit.
 - Under each quality attribute, record specific refinements of that attribute. E.g., performance might be decomposed into “data latency” and “transaction throughput” or, alternatively, “user wait time” and “time to refresh web page.” The refinements that you choose should be the ones that are relevant to your system.
 - Under each refinement, you can then record the the business value of the candidate scenario and the technical risk of achieving it.
- When generated, utility trees make system goals more specific and concrete.
- It allows understanding relative importance of NFRs helping us to decide on priority.

ATAM Scenarios

Scenario: S12 (Detect and recover from HW failure of main switch.)

Attribute: Availability

Environment: normal operations

Stimulus: CPU failure

Response: 0.999999 availability of switch

Architectural decisions	Risk	Sensitivity	Tradeoff
Backup CPU(s)	R8	S2	
No backup Data Channel	R9	S3	T3
Watchdog		S4	
Heartbeat		S5	
Failover routing		S6	

Reasoning:

- ensures no common mode failure by using different hardware and operating system (see Risk 8)

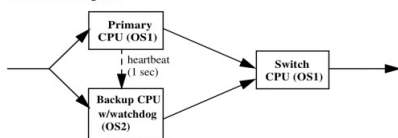
- worst-case rollover is accomplished in 4 seconds as computing state takes ...

- guaranteed to detect failure with 2 seconds based on rates of heartbeat and watchdog ...

- watchdog is simple and proven reliable

- availability requirement might be at risk due to lack of backup data channel ... (see Risk 9)

Architecture diagram:



Use cases or situations that capture stakeholders' concerns

- Link attributes, risks, trade-offs, & reasoning about architecture design
- Stakeholders vote on importance of scenarios

Qualitative

- Identify & evaluate risks

Methodical, but not exhaustive

Example from "ATAM: Method for Architecture Evaluation" (Kazman et al., 2000)

https://resources.sei.cmu.edu/asset_files/TechnicalReport/2000_005_001_13706.pdf

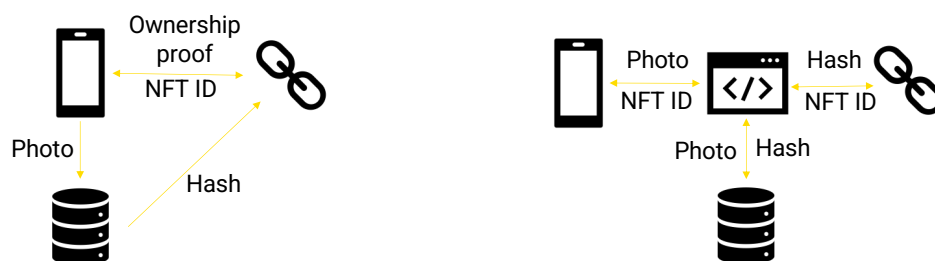
- ATAM scenarios are specific use cases or situations that capture stakeholders' concerns and illustrate how the architecture influences the system's quality attributes.
- They link attributes, risks, trade-offs, and reasoning about the architectural design and help identify potential risks or trade-offs associated with different design alternatives.
- Stakeholders vote on the importance of scenarios.
- While these scenarios are qualitative they enable the identification and evaluation of risk, typically based on scales like low, medium, and high. Then finally decisions can be made on their importance to stakeholders and associated risks.
- This is a methodical approach, but we need to make sure it is not exhaustive.

Exercise

Following are 2 proposed designs to mint an NFT for a photo you took

Compare & contrast them based on:

- Integrity
- Availability
- Cost



WK 03-01

38



- These designs depict 2 architectures for mining a non-fungible token (NFT) for a photo taken by a smartphone.
- In the 1st design, we store a photo on some off-chain storage, which will then submit the hash of the photo to the blockchain.
- Then smartphone proves the ownership of the photo to the blockchain (we can ignore specifics on how this is done) and claims the ownership of the NFT ID.
- In 2nd design, we mediate the process through a web server that abstracts blockchain interactions from the smartphone.
- The web server stores the photo on off-chain storage and stored it in hash on-chain when minting the NFT.
- Integrity – Neither design keeps the photo on the chain. Hence, integrity can be affected. However, as the photo's hash is stored on-chain, at least we can detect if the photo is modified. Hence, we can consider integrity to be moderate.
- Availability – In terms of creating and verifying the NFT, the 1st design has higher availability compared to the 2nd one because the web server has lower availability compared to the blockchain. When it comes to the availability of the photo both designs lead to a single point of failure.

- Cost – The second design has a lower transaction cost compared to the first one. This is because the second design has only a single transaction to mint the token, while the first design has two separate transactions, one to record the hash on chain and another transaction to prove ownership and mint the token. If we consider component cost, 2nd design is higher as it has an additional web server.